

UNIVERSIDAD POLITÉCNICA DE MADRID

GRADO EN INGENIERÍA BIOMÉDICA /
GRADO EN INGENIERÍA DE TECNOLOGÍAS Y
SERVICIOS DE TELECOMUNICACIÓN

ALGORITMOS Y ESTRUCTURAS DE DATOS /
ANÁLISIS Y DISEÑO DE SOFTWARE

Ejercicios de repaso de programación

Profesor:

Rodrigo García Carmona

rodrigo.garcia@upm.es

Versión 1.3



1. Introducción

A continuación encontrará los enunciados de una serie de ejercicios que le servirán para practicar. Estos ejercicios están clasificados por categorías.

Para cada ejercicio, aunque no se especifique, no deje de tener en cuenta que:

- Debe considerar los casos límite o problemáticos como arrays vacíos, valores `null`, índices fuera de rango...
- El código debe ser eficiente: si es posible evitar un bucle, ahorrar vueltas dentro de un bucle o reducir el número de líneas a ejecutar, es mejor hacerlo.
- Lo programado debe ser legible, intentando que el código sea fácil de entender y acompañándolo de comentarios completos pero concisos.
- Puede organizar su código en paquetes y clases, así como reutilizar métodos ya creados. Un buen método debería caber en una única pantalla.

Además, no deje de probar su código. No se limite a ver que compila o que funciona con una entrada de datos trivial.

2. Tipos de datos y sentencias de control

Para cada uno de los ejercicios siguientes, escriba un programa que...

1. Defina variables que representen el número de días de un año, el número de horas que tiene un día, el número de minutos que tiene una hora y el número de segundos que tiene un minuto. Emplee las variables que ocupen el mínimo espacio de memoria posible. A continuación, calcule el número de segundos que tiene un año y almacene el valor de dicho cálculo en otra variable.
2. Muestre por pantalla los mayores números enteros que se pueden representar mediante un `char`, un `short` y un `int`.
3. Calcule la suma de todos los múltiplos de 5 comprendidos entre 1 y 100. También debe calcular cuántos hay y mostrarlos.
4. Calcule el mínimo y el máximo de una serie de números enteros positivos introducidos por el usuario. Cuando el usuario introduzca un número negativo se considerará que el anterior a este es el último número.
5. Muestre por pantalla la tabla de multiplicar de los 10 primeros números naturales.
6. Muestre por pantalla la lista de los 100 primeros números primos.
7. Tome un número entero escrito por el usuario y lo descomponga en factores primos. Por ejemplo: $40 = 2 \times 2 \times 2 \times 5$.
8. Solicite al usuario dos vectores tridimensionales e imprima su producto escalar.

3. Arrays

Para cada uno de los ejercicios siguientes, escriba un programa que...

1. Tome un array de `floats` y devuelva la suma de todos sus elementos.
2. Busque un valor específico en un array de `ints` (no ordenado) y devuelva su posición, o -1 si el valor no existe.
3. Genere un array de 300 enteros en el que cada posición contenga su índice módulo 16. Es decir, que siga la siguiente secuencia:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0, 1, 2, ...

4. Haga lo mismo que el ejercicio anterior, pero ahora usando un array de **chars** con la representación en hexadecimal de los dígitos. Es decir, que siga la siguiente secuencia:

0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F, 0, 1, 2, ...

5. Dado un array de **floats**, devuelva el promedio de todos ellos.
6. Reciba una matriz (array bidimensional de **floats**, siguiendo el orden de primero fila y luego columna), y devuelva un array que contenga el valor máximo de cada fila.
7. Copie los elementos de un array de **ints** a otro, pero multiplicando cada valor por un factor fijo representado en forma de **int**.
8. Cuente el número de valores negativos que hay en un array de **floats**.
9. Invierta el orden de los elementos de un array de **chars**.
10. Haga lo mismo que el ejercicio anterior, pero sin usar arrays auxiliares. Solo puede emplear el array que recibe como argumento.
11. Pida al usuario números enteros hasta que se introduzca el número 0. A continuación, calcule la media, el mínimo y el máximo de los datos introducidos.
12. Concatene dos arrays de **chars**.
13. Divida un array en segmentos (otros arrays) de un tamaño fijo especificado a través de un entero. Si el último segmento no llega al tamaño especificado, tendrá el tamaño inferior correspondiente. Por ejemplo, si debe dividir un array de 23 elementos en segmentos de tamaño 5, acabará teniendo 4 arrays de 5 elementos y 1 array de 3 elementos.
14. Normalice un array de **floats** para que todos los valores estén entre -1 y 1. Recuerde que normalizar es dividir cada uno de los valores por el valor máximo (en valor absoluto).
15. Reciba un array de **floats** y devuelva otro array que contenga solo aquellos elementos del array original que sean más grandes que su elemento inmediatamente anterior y su elemento inmediatamente posterior. Por ejemplo, este nuevo array contendrá el elemento de la posición 3 solo si este número es mayor que el de las posiciones 2 y 4.
16. Rellene una matriz cuadrada (las dimensiones de la matriz serán un parámetro que se pida al usuario) con números aleatorios, de tal modo que la matriz sea simétrica. También debe mostrar la matriz por pantalla.
17. Implemente un filtro por umbral. Este filtro recibe un array de **floats** y devuelve otro que solo contenga los elementos que superen un umbral dado (también en forma de *float*).
18. Implemente un filtro de ventana deslizante. Este filtro recibe un array de **floats** y calcula la media móvil con una ventana de tamaño de un **int** que se pasa como argumento. Por ejemplo, la media móvil con una ventana de tamaño 3 tomará el elemento actual del array y los dos anteriores para calcular la media. La media móvil se devolverá como un array cuyo tamaño sea el del array original menos (el tamaño de la ventana menos 1) (ya que, por ejemplo, con una ventana 3 no se podrán calcular medias para los dos primeros valores del array).
19. Implemente un proceso de submuestreo que recibirá un array de **ints**, que representa un conjunto de muestras, y un **int** que representa el nivel de submuestreo; es decir, cada cuántas muestras conservará una (por ejemplo: con un nivel de submuestreo 3, se conservará 1 de cada 3 muestras). El programa deberá devolver otro array de **ints** que contenga el resultado de realizar el submuestreo.
20. Implemente un proceso de interpolación que recibirá un array de **ints**, que representa un conjunto de muestras. El programa deberá devolver otro array de **ints** que contendrá todas las muestras del array inicial y que, además, entre cada dos muestras, habrá introducido una nueva con la media de estas dos.
21. Calcule, a partir de un array de **floats**, cuántos de sus elementos están más de 2 desviaciones estándar desviados de la media de todos los elementos del array.

22. Repita el ejercicio anterior, pero logrando que su programa permita usar tanto el punto (".") como la coma (",") como separador de decimales.
23. Multiplique dos matrices de enteros. Sus dimensiones y valores deben de solicitarse al usuario por teclado y tras realizar la multiplicación debe visualizarse en pantalla ambas matrices y el resultado de la multiplicación.

4. Manejo de Strings

Para cada uno de los ejercicios siguientes, escriba un programa que...

1. Imprima por pantalla el primer número que aparezca en un **String**.
2. Indique cuántas veces aparecer un **char** en un **String**.
3. Elimine los espacios en blanco al final de un **String**.
4. Pase un **String** a TODO MAYÚSCULAS. Luego, a todo minúsculas.
5. Indique cuántas veces aparece un **String** dentro de otro **String**.
6. Reemplace puntos (".") por comas (",") en un **String**, y viceversa.
7. Verifique que un **String** contiene un número con decimales válido. Se aceptarán tanto puntos (".") como comas (",") como separador decimal.
8. Tome una cadena de caracteres (que podrá contener cualquier carácter a excepción del salto de línea) y devuelva cuántas vocales contiene.
9. Tome una cadena de caracteres (que podrá contener cualquier carácter a excepción del salto de línea) y la escriba al revés.
10. Lea una cadena de caracteres de teclado e indique si es o no palíndroma (se lee igual de izquierda a derecha que de derecha a izquierda, sin tener en cuenta los espacios en blanco y las mayúsculas). Por ejemplo: "dábale arroz a la zorra el abad".

5. Conjuntos y listas

Vuelva a programar todos los ejercicios de la sección 3, pero ahora usando **Lists** o **Sets** en lugar de arrays.

Además de lo anterior, para cada uno de los ejercicios siguientes, escriba un programa que...

1. Cree un **Set** de **Measurements** (**Measurement** es una clase que contiene un **float[]** con 4 elementos) y añada 10 **Measurements** introducidos por el usuario.
2. Modifique la clase **Measurement** para añadir un método **toString()**.
3. Modifique la clase **Measurement** para que utilice una **List** de **floats** en lugar de un **float[]**. Modifique todo el código de la clase que sea necesario. Tenga en cuenta que, a diferencia del **float[]**, la **List** podrá contener cualquier número de **floats**, no necesariamente 4.
4. Busque en un **Set** de **Measurements** todos los **Measurements** que contengan al menos un **float** cuyos valores se encuentren entre dos **floats** proporcionados por el usuario y muestre estos **Measurements** por pantalla. Tenga en cuenta que esta clase **Measurement** debería ser la que modificó en los dos ejercicios anteriores.
5. Busque **Measurements** igual que en el ejercicio anterior, pero elimine los que cumplan el criterio en lugar de mostrarlos en pantalla.
6. Convierta un **Set** de **Measurements** en un array, y viceversa.
7. Combine dos **Sets** de **Measurements** en un tercero.
8. Elimine los elementos duplicados de las **Lists** de **floats** de un **Set** de **Measurements**.

6. Manejo de archivos

Para cada uno de los ejercicios siguientes, escriba un programa que...

1. Lea un archivo de texto línea por línea y muestre el contenido por pantalla.
2. Almacene un array de `floats` en un archivo, sobrescribiéndolo.
3. Añada 3 líneas de texto escritas por el usuario al final de un archivo.
4. Copie el contenido de un archivo a otro.
5. Cuenta las líneas de un archivo.
6. Comprueba si un archivo existe antes de leerlo.
7. Cree un directorio (también llamado carpeta).
8. Liste todos los archivos con la extensión “.txt” de un directorio.
9. Obtenga el tamaño de un archivo en bytes.
10. Combine el contenido de dos archivos en un tercero, concatenándolos.

7. Programación orientada a objetos

Para la mayoría de los siguientes ejercicios deberá decidir usted mismo el tipo de dato a utilizar en los atributos de las clases. Escoja el que sea más apropiado.

1. Cree una clase para representar un punto en tres dimensiones.
Proporcione un constructor que inicialice los valores del punto al origen de coordenadas y otro que permita especificar las coordenadas del punto.
Sobrescriba el método `toString()` para que muestre información sobre los puntos.
Use la clase en un programa en el que se creen objetos que representen los puntos (12, 13, 18) y (8, 14, 0) y los muestre en pantalla de forma numérica (no gráfica).
2. Cree una clase `MyDate` que almacene el día, el mes y el año de una fecha.
Proporcione métodos para acceder a estos atributos, los llamados métodos *getters* (`getDay()`, `getMonth()` y `getYear()`), y para modificarlos, los llamados métodos *setters* (`setDay(int day)`, `setMonth(int month)` y `setYear(int year)`).
Sobrescriba el método `toString()`.
Cree la fecha 19/09/2025 y muéstrela por pantalla. Después, cambie el año a 2026 y muéstrela de nuevo por pantalla.
3. Cree una clase que represente un número racional que permita, al menos, sumar, multiplicar y simplificar números racionales.
Proporcione un constructor por defecto, un constructor de copia (esto es, un constructor al que se le pasa un objeto de la clase número racional y crea otro número racional idéntico), y otro que permita indicar los valores del numerador y del denominador por separado.
Usando esta clase, cree una calculadora que permita operar con números racionales, seleccionando las operaciones de un menú.
4. Programe una clase llamada `Person` con atributos que representen su nombre, edad, DNI, sexo (use un `enum`), peso y altura. Cree métodos para acceder y modificar todos los atributos excepto el DNI, al que solo deberá permitirse el acceso, no la modificación.
Por defecto, todos los atributos menos el DNI tendrán valores según su tipo (0 en el caso de números, una cadena vacía en el caso de `Strings`, etc.). Sexo será “mujer” por defecto.
La clase deberá tener los siguientes constructores:
 - Un constructor por defecto sin argumentos. Como el DNI no se puede modificar más tarde, este constructor generará uno cualquiera usando el constructor sin argumentos de la clase `DNI`.

- Un constructor que reciba el nombre, la edad, el DNI y el sexo.
- Un constructor que reciba todos los atributos.

La clase deberá tener los siguientes métodos:

- `computeBMI()`: calcula el índice de masa corporal (*body mass index*, BMI) de la persona. La fórmula es la siguiente: $\text{peso}/(\text{altura}^2)$. La altura deberá estar especificada en metros y el peso en kilogramos.
- `evaluateWeight()`: devuelve un enum que indica si el individuo está por debajo de su peso ideal, en su peso ideal o tiene sobrepeso. Sobrepeso se define como un BMI mayor a 25, y se considera que alguien está por debajo del peso ideal si su BMI es menor a 18.
- `isAdult()`: indica si es mayor de edad, devolviendo un `boolean`.
- `toString()`: devuelve toda la información de la persona como una cadena de caracteres.
- Métodos *getter* de cada atributo, y métodos *setter* de cada atributo excepto el DNI.

El DNI deberá estar representado a través de una clase DNI, que deberá contener:

- Un constructor que recibe el número. La letra se generará automáticamente, [siguiendo la fórmula oficial](#), a partir del número.
- Un constructor que recibe el número y la letra. No se deberá permitir crear un DNI cuya letra no sea compatible con el número.
- Un constructor que recibe un `String` que contiene el número y la letra, uno tras otro y sin separación entre ellos. No se deberá permitir crear un DNI cuya letra no sea compatible con el número.
- Un constructor por defecto sin argumentos, que genere un DNI cualquiera que sea válido.

Esta clase, además, deberá contener el método `toString()`, que devolverá una cadena de caracteres que representará el DNI.

Por último, cree una clase que se pueda ejecutar que haga lo siguiente, en orden:

- a) Pida al usuario que escriba el nombre, edad, sexo, peso y altura de tres personas, así como el número de DNI de aquellas que vaya a crear con un constructor que lo reciba. Deberá aceptarse simplemente el número, el número y la letra, o nada (el número de DNI se generará usando el constructor sin argumentos de esa clase).
 - b) Cree 3 objetos de la clase `Person`. Utilice tres tipos de constructores distintos y emplee los métodos *setters* para asignar los valores que posea y que el constructor no haya asignado ya.
 - c) Para cada persona, compruebe si está en su peso ideal, tiene sobrepeso o está por debajo de su peso ideal y comunique esta información a través de un mensaje personalizado.
 - d) Escriba un mensaje en el que se indique cuáles de las personas son mayores de edad.
 - e) Para terminar, muestre por pantalla la información completa de cada persona.
5. Programe una clase `Password` que tenga los atributos `longitud` y `password`. Si no se especifica una `longitud` al crear un objeto de la clase `Password`, esta deberá ser 8. Los constructores serán los siguientes:
- Un constructor por defecto sin argumentos que tendrá como contraseña “password”.
 - Un constructor al que se le proporcionará una longitud y que generará una contraseña alfanumérica aleatoria (investigue `Math.random`) con esa longitud.

Los métodos de esta clase serán:

- `isStrong()`: devuelve un `boolean` que indica si la contraseña es fuerte o no. Para que sea fuerte debe tener más de 2 mayúsculas, más de 1 minúscula y más de 5 números.

- `generatePassword()`: genera una contraseña para el objeto con la longitud que se especificó al crearlo.
- Métodos *getter* para `longitud` y `password`.
- Método *setter* para `longitud`.

Ahora, cree una clase ejecutable que haga lo siguiente:

- a) Cree un array de `Passwords` con el tamaño que el usuario indique.
- b) Cree un bucle que cree un nuevo objeto `Password` para cada posición del array. El usuario deberá indicar la longitud de cada contraseña.
- c) Cree otro array de `booleans` donde se almacene si el `Password` que se encuentra en la misma posición en el array de `Passwords` es o no fuerte.
- d) Muestre por pantalla cada contraseña y si es o no fuerte, siguiendo este formato:

```
contraseña1 : valor_booleano1
contraseña2 : valor_booleano2
```

6. En una empresa todos los trabajadores tienen un sueldo base de 1000 €. Los jefes tienen un suplemento de 500 € por cada año que hayan sido jefes de la empresa, y los viajantes, además del sueldo base, cobran 300 € por viaje realizado.

Cree una clase `Empleado` de la cual deriven las clases `Jefe` y `Viajante`.

Creae una plantilla de una empresa con 2 jefes, 5 viajantes y 15 empleados, y muestre por pantalla sus respectivos salarios. Para determinar el número de viajes de los viajantes y la antigüedad de los jefes puedes generar números aleatorios entre 0 y 10. Emplee el polimorfismo de herencia.

7. Cree la clase `CuentaBancaria`, que deberá tener como atributos un nombre de titular, una fecha de apertura, un número de cuenta y un saldo (este último un `float`). La cuenta deberá tener un método para retirar dinero, otro para ingresar dinero y otro para transferir dinero a otra cuenta. De una cuenta no se podrá retirar nunca dinero (ni transferir) si la cantidad de dinero a retirar o transferir es mayor que el saldo.

Cree una cuenta a plazo fijo, en la cual cuando se retira dinero de algún modo antes de una fecha de vencimiento (que será otro atributo de esta clase), además del dinero a retirar, se penaliza con un 5 % adicional.

Cree además una cuenta VIP, que tendrá un atributo adicional que es el saldo negativo máximo que puede tener. En las cuentas VIP uno podrá tener saldo negativo siempre que no se supere este valor.

Por último, programe un clase con un método `main` que permita crear los tres tipos de cuentas, transferir dinero de unas a otras, ingresar dinero y retirar dinero. Almacene las cuentas en un array o una `List` y emplee polimorfismo de herencia.

8. Manejo de excepciones

1. Convierta un array de `Strings` con números con decimales en un array de `floats`, manejando errores.
2. Divida una línea de un [archivo CSV](#) (con los valores separados por comas) en sus componentes usando `split()`. Si una línea no sigue el formato “valor1, valor2, valor3, ...”, el programa debe lanzar una excepción.
3. Tome la clase que programó para el ejercicio 2 de la sección 7 (`Mydate`), y modifíquela para que los valores introducidos para sus atributos (ya sea mediante constructores o mediante métodos *setters*) se correspondan con una fecha posible. En caso de no ser válida, se lanzará una excepción.

Cree la fecha 31/02/2025 y verifique que se lanza la excepción correspondiente. Compruebe que esto también sucede al invocar el método `setDay(35)`.

4. Pida al usuario que escriba números enteros y utilice `NumberFormatException` para, en vez de mostrar un error horrible en rojo y terminar el programa si la entrada no es válida, pedirle que, por favor, vuelva a introducir un número.
5. Incluya en cualquiera de los métodos `main` que ha ido programando hasta ahora una cláusula `finally` que capture cualquier excepción que se haya propagado hasta el `main` y, en lugar de mostrarla, muestre al usuario un mensaje pidiendo perdón humildemente.
6. Cree una clase propia llamada `InvalidMeasurementException` que extienda `Exception` y se use para cualquier error relacionado con los `Measurements` de los ejercicios de la sección 5.

Enhorabuenba, ha hecho un montón de ejercicios. Descanse, se lo ha ganado.